

UrbanCart Solution Blueprint

A 24/7 Chat & Voice Support Layer, Built on UrbanCart's Own Data

Task 1 — Discovery Analysis and Proposed Architecture

Client: UrbanCart — E-Commerce, Pakistan
Stakeholder: Sarah Malik, Operations Manager

Prepared by: Ayesha Ejaz
Solutions & Automation Engineer

Source: Discovery meeting, 1 September 2026

1. Where UrbanCart Stands Today

UrbanCart sells electronics, accessories, home, and lifestyle products through its website, Instagram, and WhatsApp, and its support team currently answers every question by hand — availability, pricing, delivery timelines, returns, and warranty terms. That works while volume is manageable, but it breaks down in two specific ways: nothing happens after hours, so a customer messaging at night simply waits until the next working day, and the records that would let the team work efficiently — customer details, order history, leads, complaints — live in separate spreadsheets and systems rather than one place. The result is lost customers overnight and a sales team that, in the client's own words, is "starting from zero" on every lead because there is no shared record of what a customer has already asked.

Two further constraints shape the design: policy and product information changes often enough that the client explicitly does not want an assistant guessing from memory, and any update to that information currently requires manual, developer-level work — something the client wants removed from the picture entirely.

2. Design Principle: Automate the Routine, Protect the Sensitive

The guiding rule behind every decision in this blueprint is simple: automation takes on anything repetitive and well-defined, and a person stays in control of anything involving judgement, money, or emotion.

Left to Automation	Kept With UrbanCart's Team
Answering availability, price, delivery, return, and warranty questions from real documents	Damaged-item reports and non-standard refund decisions
Reading back the status of an order a customer already placed	Any exchange where the customer sounds upset or escalated
Turning an interested visitor into a structured, logged lead	Closing the sale and negotiating terms
Deciding which events are worth a Slack ping	Any question the assistant isn't confident its documents actually answer
Re-indexing knowledge the moment a document changes	Approving a change to policy or escalation rules

3. Who Actually Uses This System

- **Customers**, reaching UrbanCart through the website, WhatsApp, Instagram, or a phone call, at any hour.
- **Sales staff**, who need a lead the moment it's captured, with enough context to not start the conversation cold.
- **Support/operations staff**, who need a queue of open issues with priority already attached.
- **Management**, who currently has no single place to see how leads, orders, and issues are trending.
- **Whoever maintains UrbanCart's documents**, who should be able to update the assistant's knowledge by doing nothing more than replacing a file.

4. Four Ways a Conversation Can Go

Regardless of whether it starts as a chat message or a phone call, every interaction falls into one of four shapes:

A question about a product or policy

"Do you deliver to Lahore?" or "Can I return headphones after 10 days?" — the system searches UrbanCart's actual documents for the answer and responds only with what it finds there; if nothing relevant turns up, it says so rather than inventing a policy.

A check on an order already placed

The customer supplies an order number, or enough detail to be matched to one, and receives the current status directly — no waiting for an agent to search manually.

Interest in buying

Once the conversation signals purchase intent, the assistant gathers the specifics UrbanCart asked for — name, phone, product, budget, location, and how close they are to buying — and hands a complete lead to sales instead of a half-finished one.

A complaint or anything that needs a person

Damaged goods, a complicated refund, or simple frustration are recognised early and handed to a human immediately, with the case already logged and flagged rather than the assistant attempting to talk the customer down.

5. The Data Backbone

Every category of information UrbanCart deals with is given a specific home, chosen for how that data is actually used rather than by default:

Information	Lives In	Because
Customer profiles	PostgreSQL (via Supabase)	Each customer needs one canonical record linking their orders and conversation history.
Product catalogue	PostgreSQL table + vector store	Price, stock, and SKU-level facts are structured and change often, so they're queried directly; descriptive spec text is embedded separately for RAG so wording-level questions still work.
Orders	PostgreSQL (via Supabase)	Needs a reliable join back to the customer and fast lookup by order ID during a live conversation.
Conversation history	PostgreSQL (via Supabase)	Keeps every past interaction attached to the customer, so no one starts from zero.
Leads	Airtable	Sales needs a visual, filterable pipeline they can work in without touching a database.
Support issues	Airtable	Operations needs the same filterable, status-tracked view for open complaints and escalations.

Information	Lives In	Because
Internal tasks	Airtable	Sits next to leads and support issues so day-to-day action items aren't scattered elsewhere.
Source documents (catalogue, policies, guidelines)	Google Drive	Stays where UrbanCart already edits these files; this is the single upstream source the knowledge base reads from.
Internal SOPs and escalation notes	Notion	Staff-only reference material, kept deliberately separate from anything customer-facing.

6. Orchestration: n8n as the Single Switchboard

Every channel — website chat, WhatsApp, and Vapi-handled phone calls — feeds into one n8n webhook, rather than each channel having its own separate logic. The first thing that happens to any incoming message is classification into one of the four conversation shapes from Section 4, after which n8n reads or writes whichever data store in Section 5 is relevant, applies any conditional checks (is this lead high-value? is this order delayed?), and returns a response through the same channel the customer used. Keeping this logic centralised means a WhatsApp customer and a phone caller are, in effect, served by the exact same decision-making layer — nothing is duplicated per channel.

7. Teaching the Assistant: Building and Querying the Knowledge Base

This is the part of the system responsible for the client's clearest instruction: never let the assistant invent product or policy information. It works in two distinct phases.

Phase one — building the library, whenever a document changes

- UrbanCart adds or replaces a file (catalogue, return policy, shipping policy, warranty terms, support guidelines) in a designated Google Drive folder.
- n8n detects the change automatically — nobody needs to trigger anything manually.
- Text is pulled out of the PDF, Word file, or spreadsheet.
- That text is broken into small, self-contained pieces — a single policy clause, or one product's specifications — rather than left as a whole document.
- Each piece is converted into a vector embedding that captures its meaning, tagged with where it came from and when it was last updated.
- The embeddings are stored in Supabase's vector store, and any pieces from the previous version of that document are retired so the knowledge base never answers from outdated text.

Phase two — answering a real question

- The customer's question is embedded the same way the documents were.
- A similarity search finds the closest matching pieces of UrbanCart's actual documents.
- Those pieces — and only those pieces — are handed to the assistant as the basis for its answer.
- If nothing in the search comes back as a strong match, the assistant does not guess; it tells the customer it needs to check with the team, and the question is logged for follow-up.

Because Phase one runs automatically the moment a file changes, UrbanCart's stated goal — uploading a new policy document without involving a developer — is met directly.

8. Talking to Customers: Chat and Phone Side by Side

Website and WhatsApp messages arrive as text and are routed through the same n8n logic described in Section 6. Phone calls are answered by **Vapi**, which handles the speech-to-text and text-to-speech work and, mid-call, reaches into the same n8n workflows to pull a grounded answer, check an order, or log a lead — so a caller gets identical accuracy to a chat customer, just delivered by voice. When a call needs a human, Vapi closes out the automated portion, tells the caller someone will follow up, and the case is waiting for support the moment the call ends.

9. From Interest to Lead

The moment a conversation signals buying intent, the assistant collects UrbanCart's six required fields — name, phone, product, approximate budget, location, and purchase readiness — asking a follow-up question for anything missing rather than saving a half-complete record. Before writing a new lead, the system checks whether that phone number already has a history; if so, the new interest is attached to the existing record instead of creating a duplicate. Leads marked as high-value or ready-to-buy trigger a short Slack summary to sales — for example, interest in a specific model at a stated budget and city — with a link to the full record in *Airtable*; browsing-stage leads are simply logged for the sales team to work through later.

10. Tracking an Order

A customer supplying an order ID, or enough identifying detail such as phone number and approximate date, gets a direct database lookup against PostgreSQL rather than a manual search by a support agent. The status — processing, shipped, out for delivery, delivered, or delayed — is read back in plain language on the call or in chat. If no matching order is found, the assistant says so plainly and offers escalation rather than guessing at a status; if an order shows a meaningful delay, that itself qualifies as the "important order issue" notification described in Section 12.

11. When a Human Needs to Step In

Escalation is triggered by a small, specific set of conditions: a damaged product, a complicated refund, detected frustration or anger, an explicit request to speak to a person, or a knowledge search that comes back without a confident answer. Once triggered, the assistant stops attempting to resolve things itself, a flagged case is created with full context — who the customer is, which channel they used, and the conversation so far — and the customer is told plainly that someone will follow up, rather than being left with a dead-end automated reply.

12. Keeping Slack Useful, Not Noisy

UrbanCart was explicit that Slack should not turn into background noise. So instead of notifying on every interaction, the system only posts for four specific situations:

- A high-value or purchase-ready lead

- A serious complaint or escalated conversation
- A workflow or integration failure
- An order with a significant delay or flagged problem

Everything else — routine questions, standard leads, normal order lookups — is simply recorded in the relevant data store from Section 5, visible to whoever needs to check it, without interrupting the whole team.

13. Handling Failure Gracefully

- No confident answer from the knowledge base → the assistant says it needs to check with a person, instead of guessing, and the gap is logged.
- Missing lead details → the assistant asks for them before saving anything, rather than storing an incomplete record.
- A failed system call — a timeout, a broken integration, a workflow step that errors out — is itself raised to the team as a failure needing attention, so a silent gap in the data never goes unnoticed.
- Recurring gaps (the same missing field, the same failed lookup) are visible as a pattern over time, not just as one-off incidents.

14. Why Each Tool Earns Its Place

None of the technologies below were chosen because they appeared on a list — each maps to a specific job the architecture needs done.

Tool	What It's Actually Doing Here
Vapi	Turns a phone call into the same structured intent a chat message already produces, so voice isn't a second system to maintain.
n8n	The single place where every channel's requests are classified, validated, and routed — the switchboard from Section 6.
Supabase	Runs PostgreSQL for relational data and adds a vector store for embeddings, so structured and RAG data sit on one backend instead of two.
PostgreSQL	Where anything relational — customers, orders, conversation history — needs reliable joins and lookups.
Airtable	The non-technical, filterable surface where sales and operations actually work — leads, issues, and tasks.
Google Drive	Where UrbanCart already keeps and edits its source documents; the trigger point for refreshing the knowledge base.
Notion	Internal-only documentation and escalation notes, kept apart from anything customer-facing.
Slack	Carries only the four notification categories in Section 12 — nothing else.
Zapier	One standalone, business-owned rule (Section 15) that staff can see and adjust themselves, outside the core n8n logic.
RAG	The mechanism, described in Section 7, that keeps every answer grounded in UrbanCart's real, current documents.

15. The One Zapier Workflow

When a lead in Airtable is updated to "ready to purchase," Zapier picks up the change and does two things without any n8n involvement: it books a follow-up slot on the sales team's calendar, and it sends the customer a short automatic message confirming a callback is on the way. Keeping this single rule in Zapier rather than n8n is intentional — it's simple enough that UrbanCart's own staff could modify it later without touching the underlying engineering.

16. Rolling It Out in Stages

In line with the six-to-eight week window UrbanCart asked for, the build is split so the highest-impact pieces land first and everything else extends the same architecture rather than replacing it.

First release

- Chat support across website and WhatsApp, answered from real documents
- Order-status lookup
- Lead capture with Slack alerts for high-value cases
- Human escalation with a logged, flagged case
- Automatic knowledge-base updates from Google Drive

Second release

- Vapi voice receptionist for phone calls
- The Zapier follow-up workflow
- Notion-based change log for the knowledge base
- A management-facing view across leads, orders, and issues

Nothing in the first release needs to be rebuilt to support the second — voice, for instance, plugs into the same n8n logic that chat already uses, which is the point of centralising the routing the way Section 6 describes.